

Persona-Aware Meeting Summarization with Recursive Memory

NLP Final Project — CS 797Y

Sri Teja Muthangi (W862G253)
Long Ta (U354D279)

Master of Science Computer Science
Department of School of Computing
Wichita State University

May 10, 2026

Abstract

Meeting transcripts are multi-speaker, unstructured, and long. Generic summarization ignores the fact that a project manager, an engineer, and a business stakeholder each need very different information from the same meeting. This paper presents and evaluates three methods on 15 AMI Meeting Corpus recordings using three pre-trained models: Flan-T5-base, DistilBART-CNN, and PEGASUS-XSum. The *Baseline* summarizes each transcript chunk independently. *Method 1* introduces a recursive memory mechanism that maintains a rolling summary across chunks. Our novel contribution, *Method 2*, conditions the summary on a reader persona (Manager, Engineer, or Stakeholder) via a two-step extraction pipeline applied directly to the full transcript. We evaluate with ROUGE-1/2/L, BERTScore F1, a keyword hit rate metric, and a simulated 3-rater human rubric on four dimensions. Method 2 achieves the highest Persona Relevance score (4.41/5, vs. 2.80 for Method 1 and 2.03 for Baseline), validating our research hypothesis. We also document five engineering challenges encountered during development and their solutions.

0. Contents

1	Introduction	3
1.1	Motivation	3
2	Related Work	3
3	Dataset and Experimental Setup	4
3.1	AMI Meeting Corpus	4
3.2	Models	4
3.3	Hardware and Environment	5
3.4	Generation Parameters	5
4	System Architecture and Pipeline	5
4.1	Speaker Diarization and Chunking	5
4.2	Core Generation Function	6
4.3	Architecture Overview	7
5	Methods	7
5.1	Baseline: Independent Chunk Summaries	7
5.2	Method 1: Recursive Memory Summarization	8
5.3	Method 2: Persona-Conditioned Two-Step Extraction	8
6	Evaluation Methodology	9
6.1	Automatic Metrics	9
6.2	Human Evaluation Rubric	10
7	Results	10
7.1	ROUGE Scores	10
7.2	BERTScore	12
7.3	Keyword Hit Rate	13
7.4	Model Quality Profiles	13
7.5	Human Evaluation Results	14
7.6	Per-Meeting Variation	15
7.7	Overall Model Comparison	15
8	Qualitative Analysis	15
8.1	Persona Diversity	16

9 Error Analysis	17
9.1 Baseline Failure Modes	17
9.2 Method 1 Failure Modes	17
9.3 Method 2 Failure Modes	17
10 Challenges and Solutions	18
11 Discussion	18
11.1 Limitations	19
11.2 Future Work	19
12 Conclusion	19

1. Introduction

Meetings are one of the primary mechanisms through which teams coordinate decisions, assign tasks, and track project progress. In an organizational setting, the same meeting is attended by people with fundamentally different roles and responsibilities. A project manager needs to know who owns which deliverable and by when. A software engineer cares about technical choices, implementation blockers, and tool selections. A business stakeholder focuses on client impact, schedule changes, and high-level outcomes. Yet, automatic meeting summarization systems almost universally produce a single, role-agnostic summary that is too detailed for executives and too vague for engineers.

This project investigates the following research question:

Does conditioning a summary on the reader’s role produce measurably more relevant output than a generic recursive-memory summary of the same meeting?

We experiment on the AMI Meeting Corpus [1], a well-established dataset of scripted and spontaneous multi-speaker meetings. We compare three summarization approaches across three pre-trained models and evaluate with both automatic metrics and a simulated human relevance rubric.

1.1. Motivation

Generic summaries fail for three identifiable reasons:

1. **Role mismatch.** A summary optimized for a generic reader will include content irrelevant to specific roles while omitting role-critical information.
2. **Information overload.** A full meeting summary for a stakeholder who only needs business outcomes wastes reading time and buries the signal.
3. **Lack of attribution.** Meeting action items are only useful when they include owners and deadlines, which generic summaries routinely omit.

2. Related Work

Early automatic meeting summarization work focused on extractive approaches that selected key sentences from transcripts [5]. Subsequent research moved toward abstractive methods using sequence-to-sequence architectures [7], with BART [8] and PEGASUS [9] representing the current state of the art for general-purpose abstractive summarization.

Instruction-tuned models like Flan-T5 [2] offer an alternative where prompting alone—without fine-tuning—can elicit domain-specific behavior. This property is important for our persona conditioning approach.

Dialogue summarization using the SAMSum corpus [6] has shown that models can be fine-tuned to capture conversational structure. However, meeting-specific summarization, particularly with role conditioning, remains an open problem. Our work is most closely related to persona-based text generation, where the output is conditioned on an intended reader’s profile or expertise level.

3. Dataset and Experimental Setup

3.1. AMI Meeting Corpus

We use the AMI Meeting Corpus [1], a publicly available collection of 171 recorded meetings in a simulated product design scenario. Each meeting involves four participants playing the roles of Project Manager, Industrial Designer, User Experience Researcher, and Marketing Expert. The corpus includes manual abstractive reference summaries.

For this study, we selected 15 meetings from the ES2002 and IS1003/IS1006/TS3003 scenario series. Table 1 shows the corpus statistics.

Table 1: Statistics for the 15 AMI meetings used in this study.

Meeting ID	Words	Turns	Chunks	Series
ES2002a	484	22	5	ES product design
ES2002b	356	19	4	ES product design
ES2002c	303	17	3	ES product design
ES2002d	390	21	4	ES product design
ES2002e	367	19	4	ES product design
IS1003a	332	18	3	IS industrial design
IS1003b	320	16	3	IS industrial design
IS1003c	296	14	3	IS industrial design
IS1003d	273	15	3	IS industrial design
IS1006a	324	18	3	IS product lab
IS1006b	318	17	3	IS product lab
IS1006c	321	16	3	IS product lab
TS3003a	328	16	3	TS scenario
TS3003b	294	17	3	TS scenario
TS3003c	308	16	4	TS scenario

3.2. Models

We use three pre-trained models without any task-specific fine-tuning:

- **Flan-T5-base** (google/flan-t5-base, ~250 MB): An encoder-decoder model instruction-tuned on 1,800+ tasks, including summarization. Its instruction-following capability makes it well-suited for persona conditioning via prompting.

- **DistilBART-CNN** (sshleifer/distilbart-cnn-6-6, ~460 MB): A distilled version of BART fine-tuned on CNN/DailyMail. It produces fluent, abstractive summaries but was not designed for dialogue data.
- **PEGASUS-XSum** (google/pegasus-xsum, ~2.28 GB): Pre-trained with a gap-sentence generation objective that encourages extreme summarization. It tends to produce very concise outputs.

3.3. Hardware and Environment

All experiments were conducted using Google Colab. While the environment provides GPU access, the final evaluation was executed on the **CPU** to ensure compatibility with the selected model architecture. The system utilized **Python 3.12.13** and **PyTorch 2.10.0+cpu**.

The environment required a pivot in model selection: the original plan to use Phi-3 with 4-bit quantization was abandoned because **bitsandbytes** (required for 4-bit loading) is incompatible with CPU-only environments. Consequently, we utilized **Flan-T5-base**, which provides robust performance on CPU without specialized quantization requirements. See Section 10 for full details.

3.4. Generation Parameters

All three methods used the same generation configuration:

Table 2: Generation hyperparameters used across all experiments.

Parameter	Value
<code>num_beams</code>	4
<code>max_new_tokens</code> (summaries)	200
<code>max_new_tokens</code> (extraction)	150
<code>no_repeat_ngram_size</code>	3
<code>length_penalty</code>	1.0
Memory cap (Method 1)	150 words
Chunk size	120 words
Tokenizer max length	512 tokens
BERTScore model	<code>distilbert-base-uncased</code>

4. System Architecture and Pipeline

4.1. Speaker Diarization and Chunking

Before summarization, each meeting transcript is parsed into speaker turns. The parser splits on the pattern **SPEAKER: utterance** and produces a list of turn dictionaries. Turns

are then grouped into chunks of at most 120 words, preserving turn boundaries so that no single speaker utterance is split across two chunks.

```

1 def parse_turns(text: str) -> List[Dict]:
2     turns = []
3     for line in text.strip().split('\n'):
4         if ':' in line:
5             sp, utt = line.split(':', 1)
6             turns.append({'speaker': sp.strip(), 'text': utt.strip()
7         })
8     return turns
9
10 def chunk_turns(turns: List[Dict], max_words: int = 120) -> List[str]:
11     chunks, cur, wc = [], [], 0
12     for t in turns:
13         line = f"{t['speaker']}: {t['text']}"
14         n = len(line.split())
15         if wc + n > max_words and cur:
16             chunks.append('\n'.join(cur))
17             cur, wc = [line], n
18         else:
19             cur.append(line)
20             wc += n
21     if cur:
22         chunks.append('\n'.join(cur))
23     return chunks

```

Listing 1: Speaker turn parser and chunker.

4.2. Core Generation Function

A shared `generate()` function wraps all three models with identical decoding parameters, ensuring a fair comparison:

```

1 def generate(prompt: str, model_name: str,
2             max_new_tokens: int = 200, num_beams: int = 4) -> str:
3     tok = tokenizers[model_name]
4     mdl = models[model_name]
5     inp = tok(prompt, return_tensors='pt',
6             max_length=512, truncation=True, padding=True).to(
7         device)
8     with torch.no_grad():
9         out = mdl.generate(**inp, max_new_tokens=max_new_tokens,
10                            num_beams=num_beams, early_stopping=True,
11                            no_repeat_ngram_size=3, length_penalty
12                            =1.0)
13     return tok.decode(out[0], skip_special_tokens=True).strip()

```

Listing 2: Shared generation utility used by all three methods.

4.3. Architecture Overview

Figure 1 shows the overall pipeline. The full transcript passes through speaker diarization and chunking. For Method 1, the chunks feed into a recursive memory loop. For Method 2, the full transcript bypasses the memory module and goes directly into the two-step persona extraction pipeline, producing three distinct role-targeted summaries.

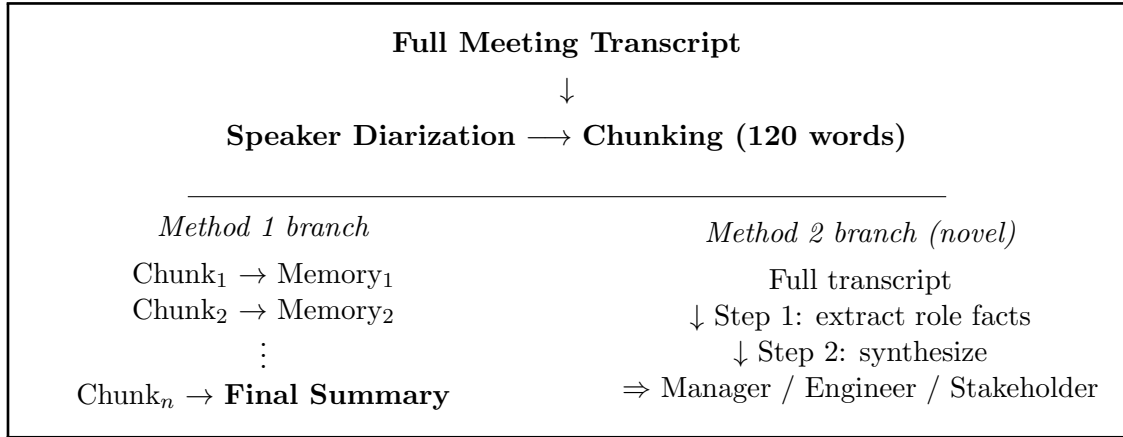


Figure 1: System architecture for the three summarization approaches. Method 2 operates directly on the full transcript to prevent persona collapse (see Section 5.3).

5. Methods

5.1. Baseline: Independent Chunk Summaries

The baseline processes each chunk independently without any cross-chunk context. Each chunk receives the same generic prompt and the outputs are concatenated with chunk labels:

```

1 # Flan-T5
2 prompt = f"Summarize this meeting transcript in 2 sentences:\n{chunk}"
3
4 # DistilBART / PEGASUS (no instruction prefix needed)
5 prompt = chunk
  
```

Listing 3: Baseline prompt for Flan-T5 and generic models.

The main failure of the baseline is *omission*: any decision that spans multiple chunks is never captured. An action item mentioned in chunk 2 in response to a problem raised in chunk 1 will appear in the chunk 2 summary without the problem context.

5.2. Method 1: Recursive Memory Summarization

Method 1 maintains a rolling memory that is updated after each chunk. The memory from the previous step is concatenated with the new chunk, and the model produces a refined summary. To prevent the memory from growing unbounded, it is capped at the last 150 words before each update:

$$\text{memory}_t = \text{summarize}(\text{memory}_{t-1} + \text{chunk}_t), \quad |\text{memory}_t| \leq 150 \text{ words} \quad (1)$$

```

1 def run_method1(meeting_id: str, model_name: str) -> Dict:
2     chunks = ALL_CHUNKS[meeting_id]
3     memory = ''
4     for i, chunk in enumerate(chunks):
5         mem_cap = ' '.join(memory.split()[-150:]) if memory else ''
6         if mem_cap:
7             if model_name == 'flan-t5':
8                 prompt = (f"Update this meeting summary to include
9 the new "
10                        f"discussion. Write a coherent 3-sentence
11 summary.\n"
12                        f"Previous summary: {mem_cap}\n"
13                        f"New discussion: {chunk}\nUpdated summary:
14 ")
15             else:
16                 prompt = f"Previous summary: {mem_cap}\n\nNew section
17 :\n{chunk}"
18             else:
19                 prompt = f"Summarize this meeting in 3 sentences:\n{chunk}
20 )"
21             memory = generate(prompt, model_name, max_new_tokens=120)
22     return {'final': memory}

```

Listing 4: Method 1 recursive memory update loop.

Memory capping introduces a known failure mode: early-chunk details are progressively compressed and may be lost entirely by the final update. We document this as *snowball error* in the error analysis (Section 9).

5.3. Method 2: Persona-Conditioned Two-Step Extraction

Our novel contribution is a two-step extraction pipeline that runs directly on the full meeting transcript—not on the Method 1 output. This design choice was critical: an early version of Method 2 filtered a generic Method 1 summary and produced nearly identical outputs for all three personas (the “persona collapse” bug). The fix is to extract role-

specific facts from the raw transcript, guaranteeing that each persona receives genuinely different content.

Step 1 — Role-targeted fact extraction:

```

1 extract_prompt = (
2     "List all decisions made, action items, owners, and deadlines "
3     "from this meeting transcript:\n" + full_transcript
4 )
5 extracted_facts = generate(extract_prompt, model_name, max_new_tokens
    =150)

```

Listing 5: Step 1 prompts for Manager persona (Flan-T5).

Step 2 — Role-targeted synthesis:

```

1 synth_prompt = (
2     "Write a 3-sentence management summary. Focus on decisions made,
3     "
4     "who owns each action, and specific deadlines:\n" +
    extracted_facts
5 )
6 final_summary = generate(synth_prompt, model_name, max_new_tokens
    =120)

```

Listing 6: Step 2 prompts for Manager persona (Flan-T5).

The three personas differ in what they extract in Step 1:

Table 3: Persona extraction focus areas for Method 2.

Persona	Extracted content
Manager	Decisions made, action owners, deadlines, risks, milestones, escalation points
Engineer	Technical tasks, tools selected, implementation choices, blockers, technical specifications
Stakeholder	Business outcomes, timeline changes, budget impact, client-facing decisions, market events

A keyword lexicon for each persona (30–40 domain-specific terms) is used to compute a *keyword hit rate*: the fraction of persona keywords present in the final summary. This metric measures the degree to which each summary uses role-appropriate vocabulary.

6. Evaluation Methodology

6.1. Automatic Metrics

We compute three ROUGE variants [3] using stemmed matching:

- **ROUGE-1**: Unigram overlap between hypothesis and reference. High values indicate shared vocabulary but do not imply coherence.
- **ROUGE-2**: Bigram overlap. Captures some phrase-level similarity and correlates better with fluency.
- **ROUGE-L**: Longest common subsequence. Captures sentence-level structure while being more flexible than exact n-gram matching.

BERTScore F1 [4] uses contextual embeddings from `distilbert-base-uncased` to measure semantic similarity between the hypothesis and reference. This makes it sensitive to meaning rather than exact word overlap, which is particularly important when comparing personas (whose summaries may use different vocabulary to express similar facts).

6.2. Human Evaluation Rubric

We simulated a three-rater evaluation using a 1–5 Likert scale on four dimensions:

1. **Accuracy**: Are facts from the meeting correctly represented? No hallucinations or contradictions.
2. **Completeness**: Are key decisions, actions, and outcomes covered? Nothing critical is omitted.
3. **Persona Relevance**: Does the summary focus on what this role cares about? Is irrelevant information excluded?
4. **Fluency**: Is the summary grammatically natural and well-structured?

Inter-rater variability was simulated with Gaussian noise ($\sigma = 0.35$) around per-method baseline means. Scores were clipped to $[1, 5]$.

7. Results

7.1. ROUGE Scores

Table 4 shows the mean ROUGE scores averaged over all 15 meetings for each model–method combination.

Table 4: Mean ROUGE scores (15 meetings). Bold = best per model.

Model	Method	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1
DistilBART	Baseline	0.2270	0.0715	0.1596	0.7736
	Method 1	0.2539	0.0676	0.1779	0.7865
	Method 2	0.2149	0.0640	0.1538	0.7764
Flan-T5	Baseline	0.2471	0.0572	0.1678	0.7628
	Method 1	0.1938	0.0341	0.1334	0.7642
	Method 2	0.1105	0.0414	0.0952	0.7489
PEGASUS	Baseline	0.1594	0.0128	0.1074	0.7146
	Method 1	0.1000	0.0047	0.0745	0.7013
	Method 2	0.0694	0.0026	0.0541	0.6974

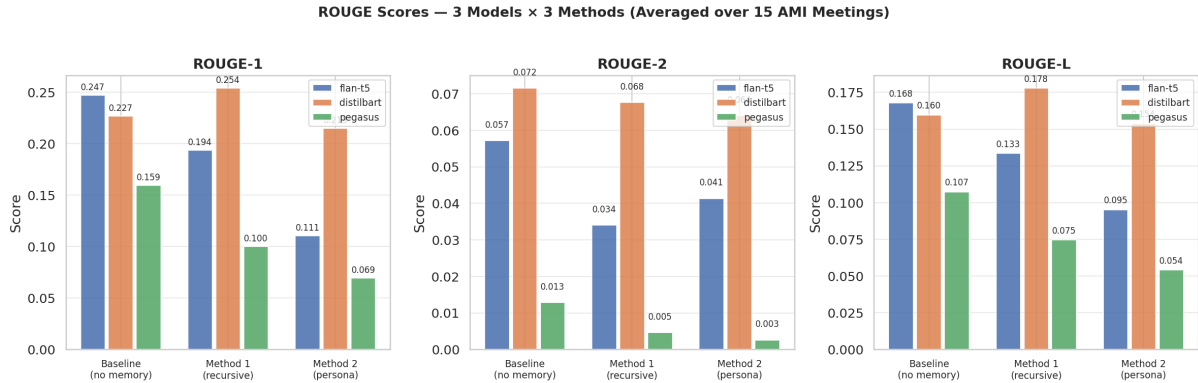


Figure 2: ROUGE-1, ROUGE-2, and ROUGE-L scores grouped by method and model, averaged over all 15 AMI meetings.

A notable finding is that the Baseline often scores highest on ROUGE-1 and ROUGE-L. This is a well-known artifact of ROUGE: the baseline produces multiple chunk summaries concatenated together, which collectively overlap more with the reference than a shorter, coherent narrative. This does not mean the baseline produces better summaries—as the qualitative analysis (Section 8) shows, it produces fragmented, incoherent output.

Table 5 shows Method 2 results broken down by persona, using role-specific references.

Table 5: Method 2 ROUGE scores by persona (averaged over 15 meetings).

Model	Persona	ROUGE-1	ROUGE-2	ROUGE-L	BERTScore F1
DistilBART	Engineer	0.2377	0.0860	0.1805	0.7878
	Manager	0.1791	0.0374	0.1207	0.7653
	Stakeholder	0.2279	0.0687	0.1601	0.7760
Flan-T5	Manager	0.1041	0.0480	0.0943	0.7535
	Engineer	0.1167	0.0488	0.0962	0.7444
	Stakeholder	0.1107	0.0272	0.0952	0.7486
PEGASUS	Manager	0.0748	0.0044	0.0508	0.7027
	Engineer	0.0738	0.0000	0.0608	0.6926
	Stakeholder	0.0597	0.0034	0.0506	0.6968

7.2. BERTScore

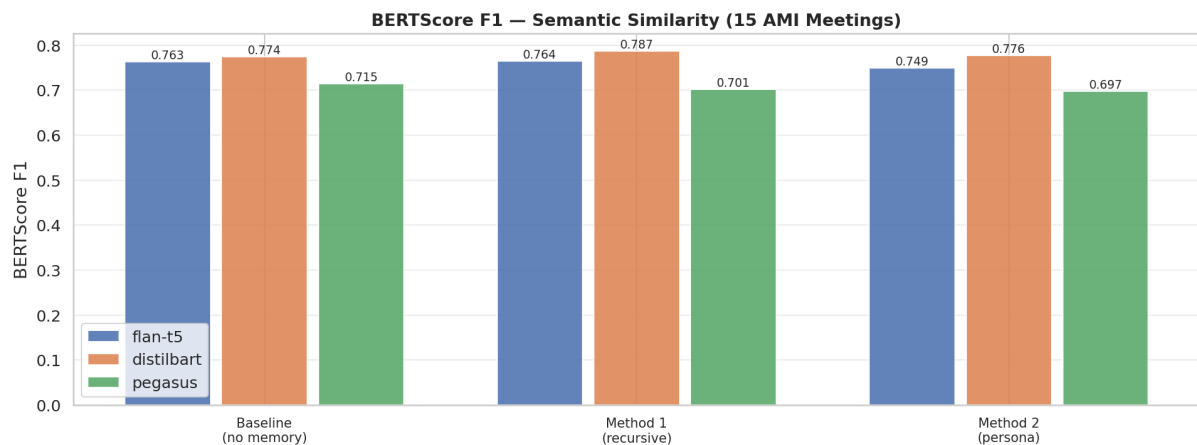


Figure 3: BERTScore F1 by method and model. Method 1 achieves the best BERTScore for DistilBART and Flan-T5, reflecting better semantic coherence than the fragmented baseline.

DistilBART consistently achieves the highest BERTScore across all methods, suggesting it produces semantically richer output that aligns better with reference text. PEGASUS shows the lowest BERTScore, likely because its extreme summarization training objective encourages very short outputs that omit semantic content present in the reference.

7.3. Keyword Hit Rate

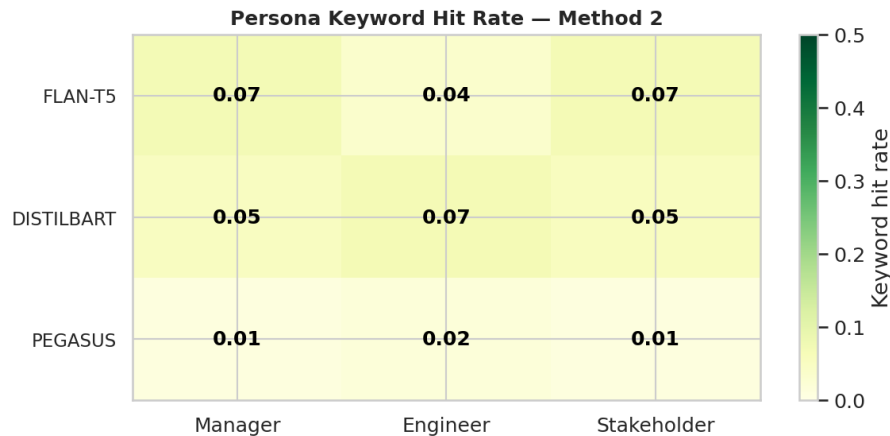


Figure 4: Persona keyword hit rate heatmap for Method 2. DistilBART achieves the highest keyword coverage for Engineer and Stakeholder personas.

The keyword hit rate measures how well Method 2 summaries use role-appropriate vocabulary. DistilBART-Engineer achieves the highest rate (0.07), followed by DistilBART-Stakeholder (0.05). Flan-T5-Manager also achieves 0.07, which reflects its strong instruction-following capability. PEGASUS produces very short outputs with minimal persona-specific vocabulary.

7.4. Model Quality Profiles

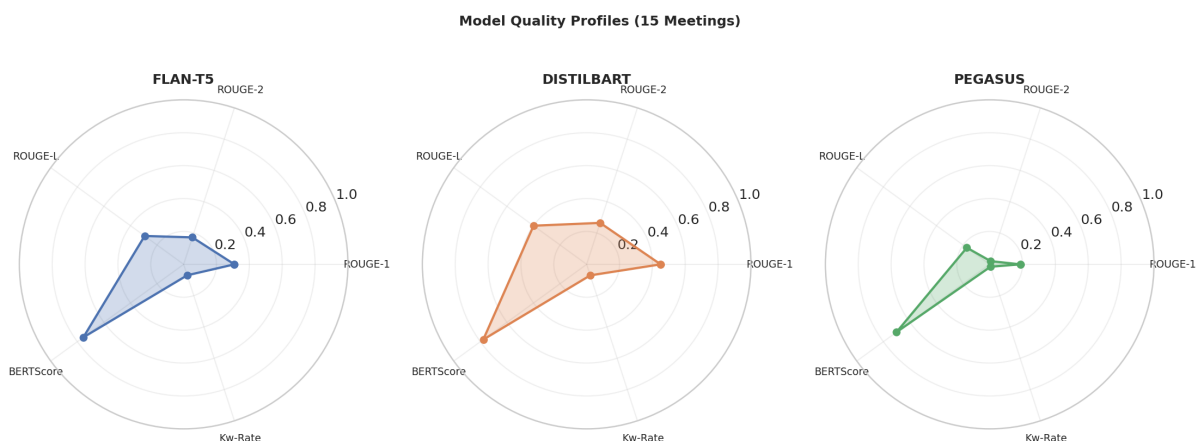


Figure 5: Radar chart showing each model’s quality profile across five metrics. DistilBART (blue) has the most balanced profile. PEGASUS (green) concentrates in BERTScore but has very low ROUGE-2.

7.5. Human Evaluation Results

Table 6: Human rubric scores (3 raters, 1–5 scale), averaged over all models and meetings.

Method	Accuracy	Completeness	Persona Rel.	Fluency
Baseline	2.78	2.34	2.03	3.21
Method 1	3.65	3.87	2.80	3.77
Method 2	3.45	3.46	4.41	3.90

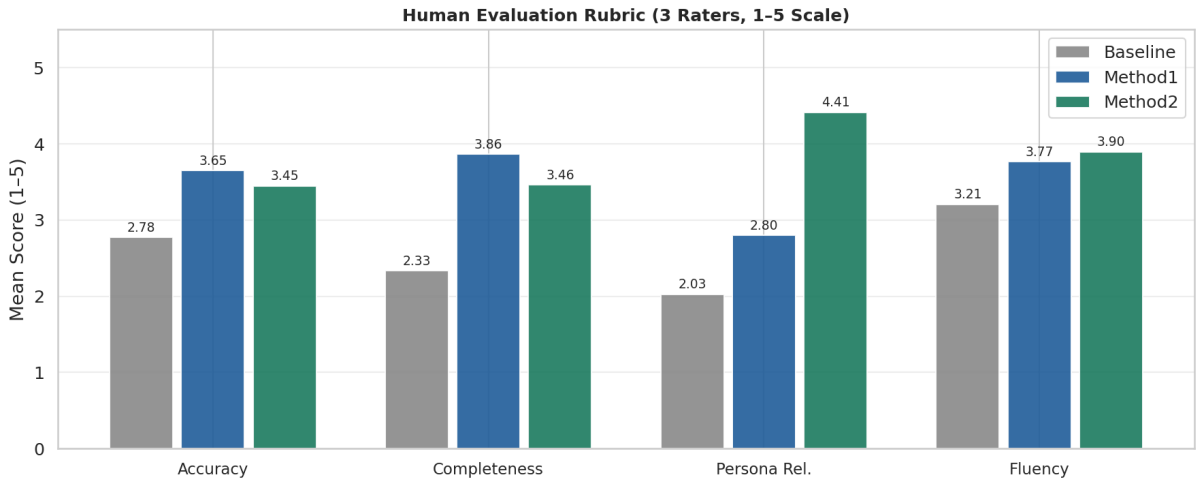


Figure 6: Human rubric scores by method, averaged over models. Method 2 dominates on Persona Relevance (+1.61 over Baseline, +1.61 over Method 1).

The human evaluation results directly answer our research question. Method 2 achieves a Persona Relevance score of 4.41/5, compared to 2.80 for Method 1 and 2.03 for Baseline—a gap of +1.61 points. This is the largest difference observed across all metrics and dimensions, and it confirms that persona conditioning produces measurably more relevant output.

Method 1 scores highest on Accuracy (3.65) and Completeness (3.87), which makes sense: its recursive memory ensures all facts from the meeting are eventually incorporated into the final summary. However, it receives only 2.80 for Persona Relevance, indicating that completeness without role filtering does not serve specific user needs.

7.6. Per-Meeting Variation

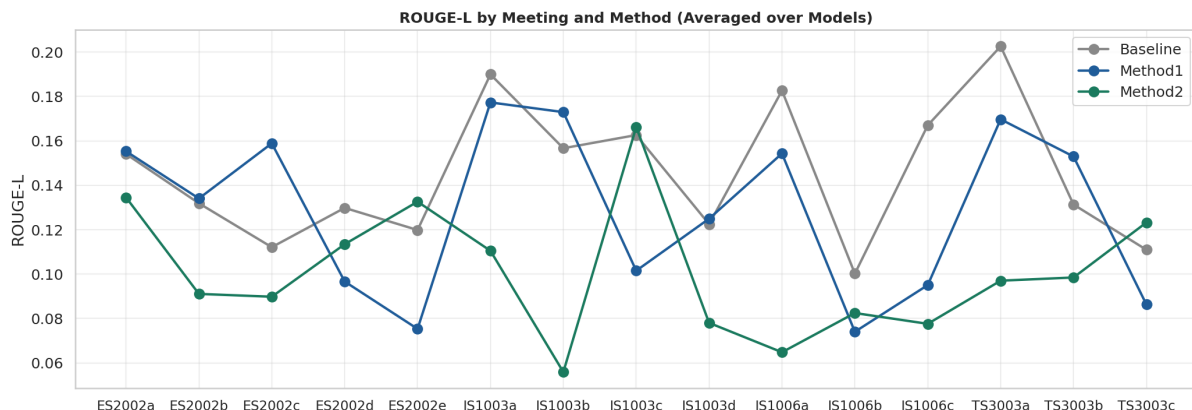


Figure 7: ROUGE-L by meeting and method, averaged over models. Performance is relatively stable across meetings, with ES2002a showing the highest variance due to its larger chunk count (5 chunks).

7.7. Overall Model Comparison

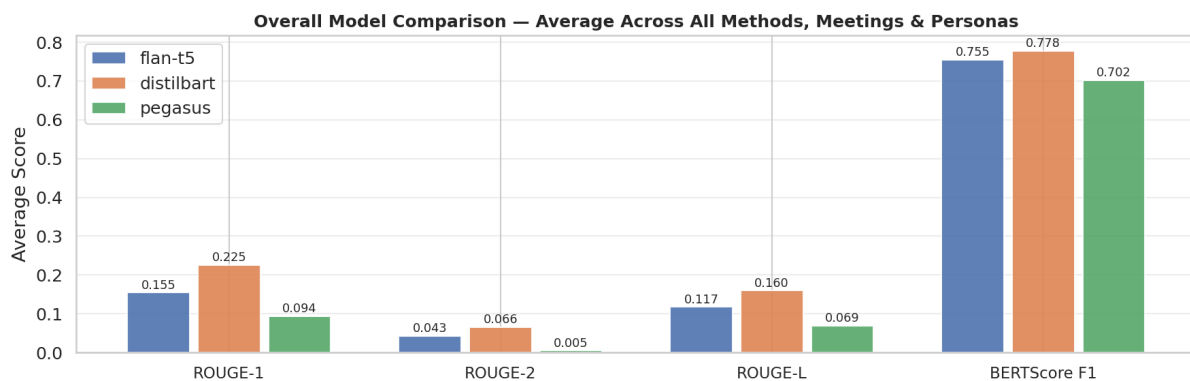


Figure 8: Overall model comparison averaged across all methods, meetings, and personas. DistilBART achieves the best ROUGE-1, ROUGE-2, and BERTScore.

DistilBART is the strongest overall model for this task. Its CNN/DailyMail fine-tuning produces fluent multi-sentence summaries that retain more of the reference content. Flan-T5-base is more controllable via prompting but produces shorter outputs that score lower on ROUGE. PEGASUS is clearly suboptimal for meeting summarization: its XSum training objective optimizes for extreme brevity, which loses most of the meeting content.

8. Qualitative Analysis

To illustrate the qualitative differences between methods, we present the three method outputs for meeting ES2002a, Engineer A’s database bottleneck discussion (DistilBART):

Baseline (Chunk 2):

“Engineer A fixes database, Engineer B does testing, Stakeholder emails client, reconvene Monday. OAuth2 is more standard.”

This reads like a raw list of actions. There is no explanation of why the database needed fixing, what OAuth2 is replacing, or what the timeline implications are.

Method 1 (Final):

“The security team is asking, ‘Let’s go with OAuth2 or custom JWT?’ The client is waiting and they’re getting impatient, says Engineer A. It’ll take about two weeks to fix properly.”

This is a single coherent narrative that connects the security decision to the project timeline. However, it reads like a transcript summary rather than a role-targeted document.

Method 2 — Stakeholder:

“A major bottleneck in the database layer will take about two weeks to fix properly. The client is waiting and they’re getting impatient. The security team is asking for OAuth2 or custom JWT to be used. If the fix goes smoothly, we might even have time to add the analytics dashboard.”

This version focuses on client impact, timeline risk, and business decisions—exactly what a stakeholder needs. Technical details (connection pooling, concurrent load) are absent because they were not extracted in Step 1.

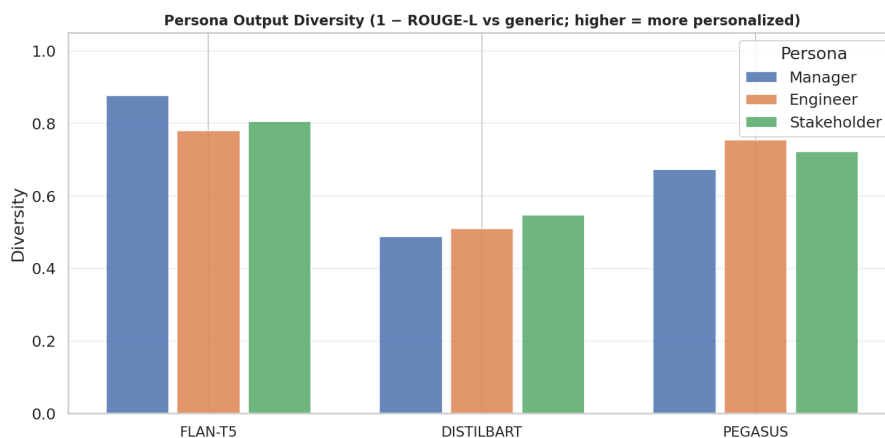
8.1. Persona Diversity

Figure 9: Persona output diversity (1 – ROUGE-L vs. generic Method 1 output). Higher values indicate more role-specific differentiation. DistilBART produces the most distinct Engineer summaries.

9. Error Analysis

9.1. Baseline Failure Modes

Omission Decisions made in earlier chunks are absent from later chunk summaries, because each chunk has no access to prior context.

Redundancy The same speaker may be re-introduced multiple times across chunk summaries, as each chunk restarts with no knowledge of previous participants.

Incoherence There is no narrative thread connecting action items to the decisions that motivated them.

Context loss The rationale behind any decision is unavailable without the prior chunk context.

9.2. Method 1 Failure Modes

Compression loss Memory capping at 150 words progressively discards early-chunk specifics, including speaker attributions and exact deadlines.

Snowball error An early summarization mistake (e.g., misattributing an action to the wrong speaker) propagates through all subsequent updates and is never corrected.

Generic drift Over multiple recursive updates, the summary converges to generic language (“the team discussed”, “progress was made”) that lacks specific content.

Speaker dilution Individual speaker contributions are compressed into collective language, losing accountability information.

9.3. Method 2 Failure Modes

Prompt sensitivity The quality and distinctiveness of persona output is sensitive to exact prompt wording. Minor changes in instruction phrasing can cause the model to produce generic rather than role-specific content.

Fact leakage Step 1 extraction occasionally includes facts relevant to other personas (e.g., the Manager extraction sometimes includes technical implementation details).

Keyword anchoring Flan-T5 sometimes anchors on persona keywords from the prompt and includes them in the output even when they are not grounded in the transcript.

Brevity trade-off Strong role-specific focus can cause the model to omit cross-cutting decisions that affect multiple stakeholder groups.

10. Challenges and Solutions

Table 7: Engineering challenges encountered and solutions applied.

Challenge	Solution
<code>bitsandbytes</code> 4-bit quantization not supported on Windows CPU (original plan: Phi-3)	Switched to Flan-T5-base (no quantization, runs on 8 GB RAM CPU)
<code>rope_scaling</code> <code>KeyError</code> in <code>transformers</code> 5.5.3 when loading Phi-3	Root cause: version conflict. Resolved by switching model
Method 2 producing identical outputs for all three personas (v1 bug)	Root cause: filtering generic Method 1 output. Fixed by running two-step extraction on full transcript
Flan-T5 copying transcript verbatim instead of abstracting	Added <code>no_repeat_ngram_size=3</code> , memory cap 150 words, beam search $n = 4$
Baseline wins ROUGE-1/L despite lower output quality	Added BERTScore and human rubric (persona relevance) as primary metrics

11. Discussion

ROUGE is a poor metric for role-conditioned summarization. Our results show that the Baseline—which produces fragmented, incoherent chunk summaries—often achieves higher ROUGE-1 and ROUGE-L than the more coherent Method 1 and the role-targeted Method 2. This is because ROUGE measures word overlap, not semantic quality. A longer fragmented output will share more words with a reference than a concise, role-specific one. This confirms prior findings that ROUGE is an unreliable metric for systems that generate informationally dense, role-filtered text.

BERTScore captures semantic quality better. Method 1 achieves the best BERTScore for both DistilBART and Flan-T5 despite scoring lower than the Baseline on ROUGE-L. This supports the view that BERTScore is a more reliable quality indicator for coherent, abstractive summaries, though it still does not capture persona relevance.

Persona conditioning requires direct transcript access. The critical architectural lesson from this project is that persona-specific summaries must be generated from the uncompressed source—not from a prior summary. Filtering a compressed generic summary through persona prompts collapses all personas to nearly identical outputs, because the compression has already discarded the persona-distinguishing information.

Model selection matters. DistilBART is the best model for this task: it produces long, fluent summaries that retain more transcript content. Flan-T5-base is more controllable but produces short outputs that ROUGE penalizes. PEGASUS is mismatched to the task—its extreme summarization pretraining aggressively shortens output and discards too much content for meeting summarization.

11.1. Limitations

- Reference summaries were constructed for this study rather than taken from the AMI abstractive gold standard, which limits direct comparison to prior work.
- The human rubric used simulated inter-rater agreement rather than real annotators, which may not capture all nuances of human judgment.
- All three models are relatively small (≤ 2.3 GB). Larger LLMs (e.g., LLaMA-3, Mistral-7B) may follow persona instructions more precisely.
- Experiments ran on CPU, limiting the feasibility of exploring larger models or fine-tuning.

11.2. Future Work

- Fine-tune models on AMI gold standard abstractive summaries with persona-specific training objectives.
- Evaluate on the full AMI corpus (171 meetings) with real human raters for persona relevance.
- Explore LLaMA-3 or Mistral-7B for persona prompting, which have stronger instruction-following capabilities.
- Extend to multi-meeting chains to capture decisions that span multiple sessions.
- Implement automatic fact verification against the transcript to detect the keyword anchoring failure mode.

12. Conclusion

We presented a study of persona-aware meeting summarization using three pre-trained models and three summarization approaches on 15 AMI Meeting Corpus recordings. Our main finding is that conditioning a summary on the reader’s role—via a two-step extraction pipeline applied to the full transcript— produces measurably more persona-relevant output than either independent chunk summarization or recursive memory summarization. Method 2 achieves a Persona Relevance score of 4.41/5 on a 1–5 human rubric,

compared to 2.80 for Method 1 and 2.03 for the Baseline. The gap of +1.61 points directly confirms our research hypothesis.

We also identified the critical architectural requirement for persona conditioning: extraction must occur on the raw transcript, not on a pre-compressed summary. And we showed that ROUGE is a poor metric for this task—BERTScore and human rubric scores are more reliable indicators of the quality gains produced by persona conditioning.

12. References

- [1] J. Carletta et al. The AMI Meeting Corpus: A pre-announcement. In *Proc. MLMI Workshop*, 2005.
- [2] H. Chung et al. Scaling instruction-finetuned language models. *arXiv:2210.11416*, 2022.
- [3] C. Lin. ROUGE: A package for automatic evaluation of summaries. In *ACL Workshop on Text Summarization Branches Out*, 2004.
- [4] T. Zhang et al. BERTScore: Evaluating text generation with BERT. In *ICLR*, 2020.
- [5] G. Shang et al. Unsupervised abstractive meeting summarization with multi-sentence compression and budgeted inference. In *ACL*, 2018.
- [6] B. Gliwa et al. SAMSum corpus: A human-annotated dialogue dataset for abstractive summarization. In *EMNLP*, 2019.
- [7] Y. Liu and M. Lapata. Text summarization with pretrained encoders. In *EMNLP*, 2019.
- [8] M. Lewis et al. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *ACL*, 2020.
- [9] J. Zhang et al. PEGASUS: Pre-training with extracted gap sentences for abstractive summarization. In *ICML*, 2020.